

CS 224N Winter 2026 Assignment 2

Word2Vec and Dependency Parsing

Due date: January 22nd, Thursday, 4:30 PM PST

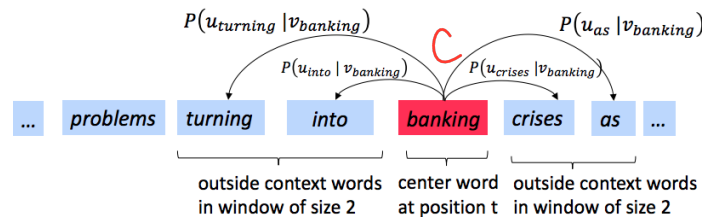
In this assignment, you will review the mathematics behind Word2Vec and build a neural dependency parser using PyTorch. For a review of the fundamentals of PyTorch, please check out the PyTorch review session on Canvas. In Part 1, you will explore the partial derivatives involved in training a Word2vec model using the naive softmax loss. In Part 2, you will learn about two general neural network techniques (Adam Optimization and Dropout). In Part 3, you will implement and train a dependency parser using the techniques from Part 2, before analyzing a few erroneous dependency parses.

If you are using LaTeX, you can use `\ifans{}` to type your solutions.

Please tag the questions correctly on Gradescope, otherwise the TAs will take points off if you don't tag questions.

1. Understanding word2vec (20 points)

Recall that the key insight behind word2vec is that *'a word is known by the company it keeps'*. Concretely, consider a 'center' word c surrounded before and after by a context of a certain length. We term words in this contextual window 'outside words' (O). For example, in Figure 1 the context window length is 2, the center word c is 'banking', and the outside words are 'turning', 'into', 'crises', and 'as':



중심 단어 이가 주어졌을 때
주변 단어 이가 나올
조건부 확률
 $P(O=o | C=c)$

Figure 1: The word2vec skip-gram prediction model with window size 2

Skip-gram word2vec aims to learn the probability distribution $P(O|C)$. Specifically, given a specific word o and a specific word c , we want to predict $P(O = o | C = c)$: the probability that word o is an 'outside' word for c (i.e., that it falls within the contextual window of c). We model this probability by taking the softmax function over a series of vector dot-products:

$$P(O = o | C = c) = \frac{\exp(\mathbf{u}_o^\top \mathbf{v}_c)}{\sum_{w \in \text{Vocab}} \exp(\mathbf{u}_w^\top \mathbf{v}_c)} \quad \text{softmax} \quad (1)$$

For each word, we learn vectors u and v , where $\mathbf{u}_o$ is the 'outside' vector representing outside word o , and $\mathbf{v}_c$ is the 'center' vector representing center word c . We store these parameters in two matrices, $\mathbf{U}$ and $\mathbf{V}$. The columns of $\mathbf{U}$ are all the 'outside' vectors $\mathbf{u}_w$; the columns of $\mathbf{V}$ are all of the 'center' vectors $\mathbf{v}_w$. Both $\mathbf{U}$ and $\mathbf{V}$ contain a vector for every $w \in \text{Vocabulary}$.¹

¹Assume that every word in our vocabulary is matched to an integer number k . Bolded lowercase letters represent vectors. $\mathbf{u}_k$ is both the k^{th} column of $\mathbf{U}$ and the 'outside' word vector for the word indexed by k . $\mathbf{v}_k$ is both the k^{th} column of $\mathbf{V}$ and the 'center' word vector for the word indexed by k . In order to simplify notation we shall interchangeably use k to refer to word k and the index of word k .

Recall from lectures that, for a single pair of words c and o , the loss is given by:

$$\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U}) = -\log P(O = o | C = c). \quad (2)$$

We can view this loss as the cross-entropy² between the true distribution $\mathbf{y}$ and the predicted distribution $\hat{\mathbf{y}}$, for a particular center word c and a particular outside word o . Here, both $\mathbf{y}$ and $\hat{\mathbf{y}}$ are vectors with length equal to the number of words in the vocabulary. Furthermore, the k^{th} entry in these vectors indicates the conditional probability of the k^{th} word being an 'outside word' for the given c . The true empirical distribution $\mathbf{y}$ is a one-hot vector with a 1 for the true outside word o , and 0 everywhere else, for this particular example of center word c and outside word o ³. The predicted distribution $\hat{\mathbf{y}}$ is the probability distribution $P(O | C = c)$ given by our model in equation (1).

Note: Throughout this homework, when computing derivatives, please use the method reviewed during the lecture (i.e. no Taylor Series Approximations).

- (a) (2 points) Prove that the naive-softmax loss (Equation (2)) is the same as the cross-entropy loss between $\mathbf{y}$ and $\hat{\mathbf{y}}$, i.e. (note that $\mathbf{y}$ (true distribution), $\hat{\mathbf{y}}$ (predicted distribution) are vectors and $\hat{\mathbf{y}}_o$ is a scalar):

$$CE(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{w \in \text{Vocab}} y_w \log \hat{y}_w = -\log \hat{y}_o = -\sum_{w \in \text{Vocab}} \mathbf{y}_w \log(\hat{\mathbf{y}}_w) = -\log(\hat{\mathbf{y}}_o). \quad (3)$$

* $\mathbf{y}$ is one-hot vector

Your answer should be one line. You may describe your answer in words.

- (b) (6 points) i. Compute the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{v}_c$. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, $\mathbf{U}$, and show your work to receive full credit.

Note: Your final answers for the partial derivative should follow the shape convention: the partial derivative of any function $f(x)$ with respect to x should have the **same shape** as x ⁴.

Please provide your answers for the partial derivative in vectorized form. For example, when we ask you to write your answers in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{U}$, you may not refer to specific elements of these terms in your final answer (such as $\mathbf{y}_1$, $\mathbf{y}_2$, ...). You may also not refer to specific vectors such as $\mathbf{u}_0$, $\mathbf{u}_1$, etc.

- ii. When is the gradient you computed equal to zero? Write a mathematical equation.

Hint: You may wish to review and use some introductory linear algebra concepts.

- iii. The gradient you found is the difference between the two terms. Provide an interpretation of how each of these terms improves the word vector when this gradient is subtracted from the word vector $\mathbf{v}_c$.

- (c) (1 point) In many downstream applications using word embeddings, L2 normalized vectors (e.g. $\mathbf{u}/\|\mathbf{u}\|_2$ where $\|\mathbf{u}\|_2 = \sqrt{\sum_i u_i^2}$) are used instead of their raw forms (e.g. $\mathbf{u}$). Let's consider a hypothetical downstream task of binary classification of phrases as being positive or negative, where you decide the sign based on the sum of individual embeddings of the words. When would L2 normalization take away useful information for the downstream task? When would it not?

Hint: Consider the case where $\mathbf{u}_x = \alpha \mathbf{u}_y$ for some words $x \neq y$ and some scalar α . When α is positive, what will be the value of normalized $\mathbf{u}_x$ and normalized $\mathbf{u}_y$? How might $\mathbf{u}_x$ and $\mathbf{u}_y$ be related for such a normalization to affect or not affect the resulting classification?

²The **cross-entropy loss** between the true (discrete) probability distribution p and another distribution q is $-\sum_i p_i \log(q_i)$.

³Note that the true conditional probability distribution of context words for the entire training dataset would not be one-hot.

⁴This allows us to efficiently minimize a function using gradient descent without worrying about reshaping or dimension mismatching. While following the shape convention, we're guaranteed that $\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$ is a well-defined update rule.

L2 정규화

① 유용한 정보 잃는 경우 : 단어의 강도 (intensity)나 빈도, 중요도 정보 사라질 때.
 ② 유용한 정보 잃지 않는 경우 : 단어의 크기 대신 방향이 결정적일 때,
 정규화 후에도 벡터의 방향은 유지되므로 영향 없음.

(e)

$$J = -u_o^T v_c + \log \sum_{x \in V} \exp(u_o^T v_c)$$

$$\frac{\partial J}{\partial u_o} = -v_c + \frac{1}{\sum_x \exp(u_o^T v_c)} \cdot \exp(u_o^T v_c) v_c$$

$$= -v_c + \hat{y}_o v_c$$

$$= (\hat{y}_o - 1) v_c$$

$$\frac{\partial J}{\partial u_w} = 0 + \frac{\partial}{\partial u_w} \log \sum_x \exp(u_o^T v_c)$$

$$= \frac{1}{\sum_x \exp(u_o^T v_c)} \cdot \exp(u_o^T v_c) v_c$$

$$= \hat{y}_w v_c = (\hat{y}_w - y_w) v_c$$

$$\therefore \frac{\partial J}{\partial u_w} = (\hat{y}_w - y_w) v_c$$

$$\frac{\partial J}{\partial U} = \left[\frac{\partial J}{\partial u_1}, \frac{\partial J}{\partial u_2}, \dots, \frac{\partial J}{\partial u_{|V|}} \right]$$

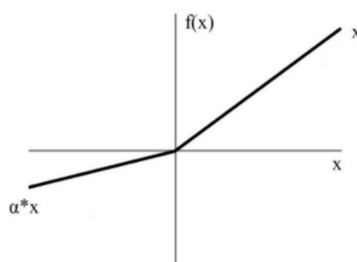
(d) (1 point) Write down the partial derivative of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to $\mathbf{U}$. Please break down your answer in terms of the column vectors $\frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_1}$, $\frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_2}$, $\dots$, $\frac{\partial \mathbf{J}(\mathbf{v}_c, o, \mathbf{U})}{\partial \mathbf{u}_{|\text{Vocab}|}}$ (do not further expand these terms). No derivations are necessary, just an answer in the form of a matrix.

(e) (5 points) Compute the partial derivatives of $\mathbf{J}_{\text{naive-softmax}}(\mathbf{v}_c, o, \mathbf{U})$ with respect to each of the 'outside' word vectors, $\mathbf{u}_w$'s. There will be two cases: when $w = o$, the true 'outside' word vector, and $w \neq o$, for all other words. Please write your answer in terms of $\mathbf{y}$, $\hat{\mathbf{y}}$, and $\mathbf{v}_c$. In this subpart, you may use specific elements within these terms as well (such as $y_1, y_2, \dots$). Note that $\mathbf{u}_w$ is a vector while $y_1, y_2, \dots$ are scalars. Show your work to receive full credit.

$$\frac{df(y)}{dy} = \begin{cases} 1 & \text{if } y = 1 \\ 0 & \text{if } y = 0 \end{cases}$$

(f) (2 points) As an additional exercise for this problem, you will be taking the derivatives of some common loss functions, which may be used in variations of word2vec (such as the negative sampling variant). The Leaky ReLU (Leaky Rectified Linear Unit) activation function is given by Equation [4] and Figure [2]:

$$f(x) = \max(\alpha x, x) \quad (4)$$



$$\frac{df(y)}{dy} = \sigma(y)(1 - \sigma(y))$$

Figure 2: Leaky ReLU

Where x is a scalar and $0 < \alpha < 1$, please compute the derivative of $f(x)$ with respect to x . You may ignore the case where the derivative is not defined at 0 [5]

(g) (3 points) The sigmoid function is given by Equation [5]:

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1} \quad (5)$$

Please compute the derivative of $\sigma(x)$ with respect to x , where x is a scalar. Please write your answer in terms of $\sigma(x)$. Show your work to receive full credit.

$$\frac{d\sigma(y)}{dy} = \sigma(y)(1 - \sigma(y))$$

⁵If you're interested in how to handle the derivative at this point, you can read more about the notion of subderivatives.